

Computational Methods in Economics - Problem Set 2

Luis Felipe Lins

March 2026

In this problem set, our goal is apply concepts of interpolation and root finding learned in topics 3 and 4 of the CME subject. The repository containing the codes used in this problem set's solution can be found on [GitHub](#).

1 Question 1

In this question, we have census results for Brazilian life expectancy data between 1940 and 2010. The goal here is to interpolate the available data points (for the beginning of each decade) in order to have estimates of life expectancy for all years in this interval. I propose doing so using a quadratic interpolation. The results can be found in the chart below. Answering to item (a) of this question, using this proposed method, my best guess is that life expectancy in 1996 was 69.3 years.

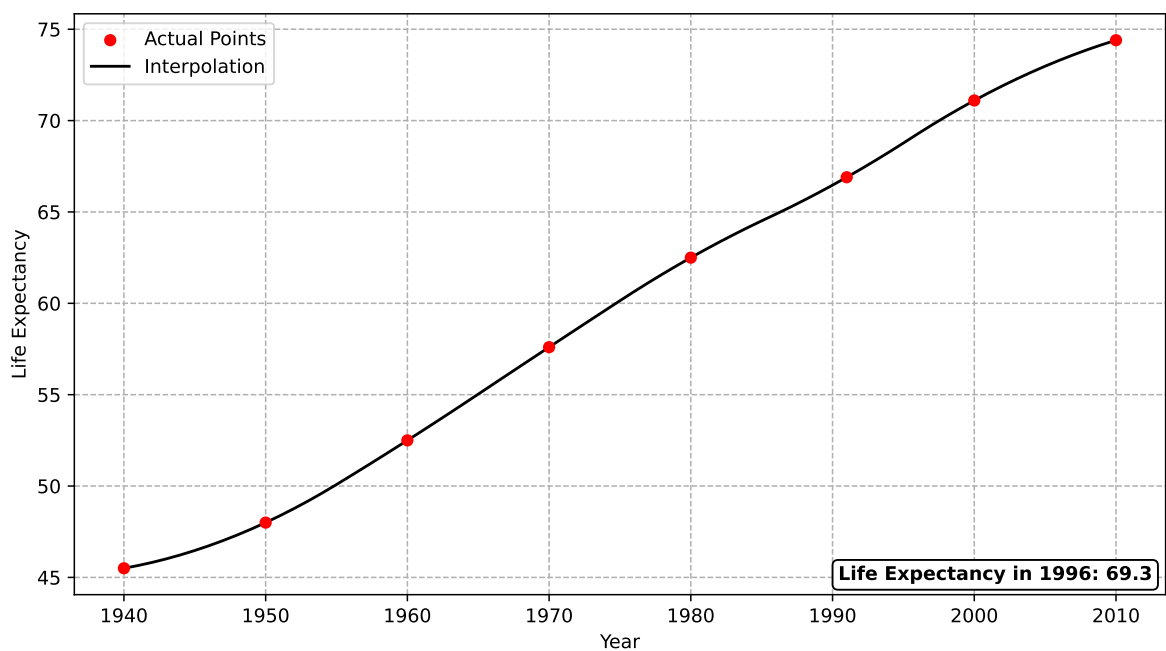


Figure 1: Quadratic Interpolation of the Brazilian Life Expectancy

2 Question 2

For this question, we're given the PDF of a Pareto distribution and are proposed to perform an interpolation exercise. Assuming we only know (observe) a number n of points of this function, we're asked to interpolate it using a linear, a spline and an akima approximation. Later, we'll compare the interpolation approximation, evaluated in 2500 points, to the actual function value at these points. First, I do this for a normal, uniformly distributed grid, and then I test a log-spaced grid too. Below, we plot a table with the results for a uniform grid, alongside a plot with the time elapsed in running each interpolation method in the X-axis, with the error in the Y-axis, where each dot denotes an observed sample size (n):

	$n = 10$	$n = 15$	$n = 20$	$n = 30$	$n = 50$
Linear ε	1.1488	0.3785	0.1415	0.0325	0.0044
Spline ε	0.3861	0.0800	0.0195	0.0021	$7.1 \cdot 10^{-5}$
Akima ε	0.4659	0.1017	0.0279	0.0037	$2.4 \cdot 10^{-4}$
Linear Δ	$6.2 \cdot 10^{-5}$	$5.2 \cdot 10^{-5}$	$6.1 \cdot 10^{-5}$	$6.5 \cdot 10^{-5}$	$7.8 \cdot 10^{-5}$
Spline Δ	$6.6 \cdot 10^{-5}$	$7.0 \cdot 10^{-5}$	$7.4 \cdot 10^{-5}$	$8.2 \cdot 10^{-5}$	$9.2 \cdot 10^{-5}$
Akima Δ	$6.5 \cdot 10^{-5}$	$7.0 \cdot 10^{-5}$	$7.3 \cdot 10^{-5}$	$8.3 \cdot 10^{-5}$	$9.1 \cdot 10^{-5}$

Table 1: Interpolation MSE (ε) and average evaluation time in seconds (Δ) by grid size.

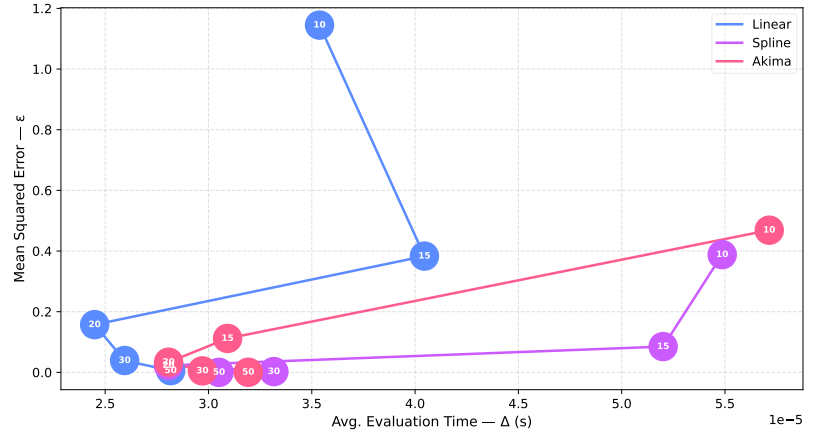


Figure 2: MSE vs. evaluation time by interpolation method.

We can also do this same analysis for a log-spaced grid:

	$n = 10$	$n = 15$	$n = 20$	$n = 30$	$n = 50$
Linear ε	0.1116	0.0218	0.0063	0.0013	$1.8 \cdot 10^{-4}$
Spline ε	0.0189	0.0013	$1.5 \cdot 10^{-4}$	$7.0 \cdot 10^{-6}$	$1.0 \cdot 10^{-7}$
Akima ε	0.0192	0.0018	$3.5 \cdot 10^{-4}$	$3.6 \cdot 10^{-5}$	$1.6 \cdot 10^{-6}$
Linear Δ	$4.2 \cdot 10^{-5}$	$5.4 \cdot 10^{-5}$	$5.6 \cdot 10^{-5}$	$6.6 \cdot 10^{-5}$	$7.2 \cdot 10^{-5}$
Spline Δ	$7.1 \cdot 10^{-5}$	$7.4 \cdot 10^{-5}$	$7.4 \cdot 10^{-5}$	$8.6 \cdot 10^{-5}$	$8.8 \cdot 10^{-5}$
Akima Δ	$6.2 \cdot 10^{-5}$	$6.8 \cdot 10^{-5}$	$9.7 \cdot 10^{-5}$	$8.0 \cdot 10^{-5}$	$9.5 \cdot 10^{-5}$

Table 2: Log-grid interpolation MSE (ε) and average evaluation time in seconds (Δ) by grid size.

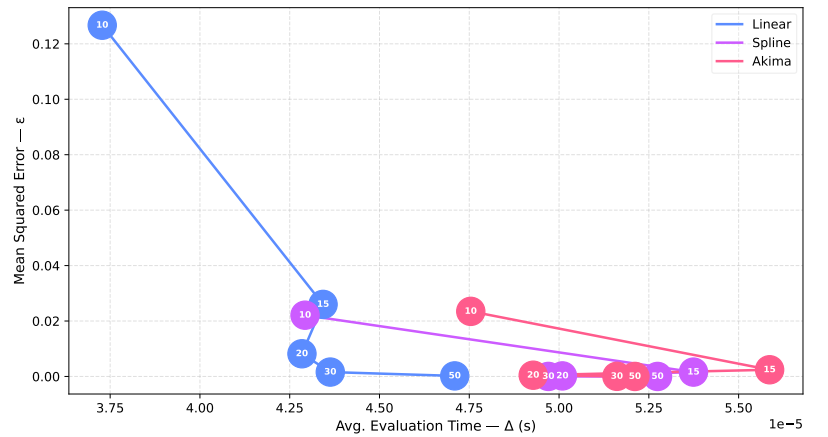


Figure 3: MSE vs. evaluation time by interpolation method (log grid).

We can take a few things from this exercise. First of all, note that even though the error is considerably higher for the linear interpolation when we have only a few points ($n = 10$), the error quickly falls when n increases, as we can approximate the function more precisely at a lower computational cost. This computational cost can be observed in the X-axis: the average iteration time. The linear interpolation is almost always considerably faster than the spline and the akima, which yield similar results both in terms of precision and duration (the spline is a little bit better in most cases). Finally, using a log-spaced grid increases precision significantly in all interpolation methods: for example, when using a log-spaced grid and $n = 10$, the approximation error of the linear interpolation is almost 4 times lower than the approximation error of both the spline and akima in the uniform-spaced grid.

3 Question 3

Now, our goal is to perform the same exercise as we did in the previous question, but instead of using a Pareto distribution PDF, we'll use the following function:

$$g(x) = [1 + \exp(-\eta \cdot \log(x/\xi))]^{-1}, \text{ with } \eta = 5 \text{ and } \xi = 500$$

The uniform- and log-spaced grid, respectively, can be found below:

	$n = 10$	$n = 15$	$n = 20$	$n = 30$	$n = 50$
Linear ε	$2.60 \cdot 10^{-5}$	$4.56 \cdot 10^{-6}$	$1.32 \cdot 10^{-6}$	$2.45 \cdot 10^{-7}$	$3.12 \cdot 10^{-8}$
Spline ε	$2.66 \cdot 10^{-7}$	$6.27 \cdot 10^{-10}$	$2.75 \cdot 10^{-11}$	$9.78 \cdot 10^{-13}$	$1.53 \cdot 10^{-14}$
Akima ε	$5.30 \cdot 10^{-6}$	$1.33 \cdot 10^{-7}$	$2.52 \cdot 10^{-8}$	$1.39 \cdot 10^{-9}$	$7.24 \cdot 10^{-11}$
Linear Δ	$5.1 \cdot 10^{-5}$	$9.6 \cdot 10^{-5}$	$5.8 \cdot 10^{-5}$	$6.6 \cdot 10^{-5}$	$8.0 \cdot 10^{-5}$
Spline Δ	$7.2 \cdot 10^{-5}$	$8.7 \cdot 10^{-5}$	$1.1 \cdot 10^{-4}$	$8.6 \cdot 10^{-5}$	$9.6 \cdot 10^{-5}$
Akima Δ	$7.3 \cdot 10^{-5}$	$9.5 \cdot 10^{-5}$	$7.8 \cdot 10^{-5}$	$9.4 \cdot 10^{-5}$	$1.1 \cdot 10^{-4}$

Table 3: Interpolation MSE (ε) and average evaluation time in seconds (Δ) by grid size.

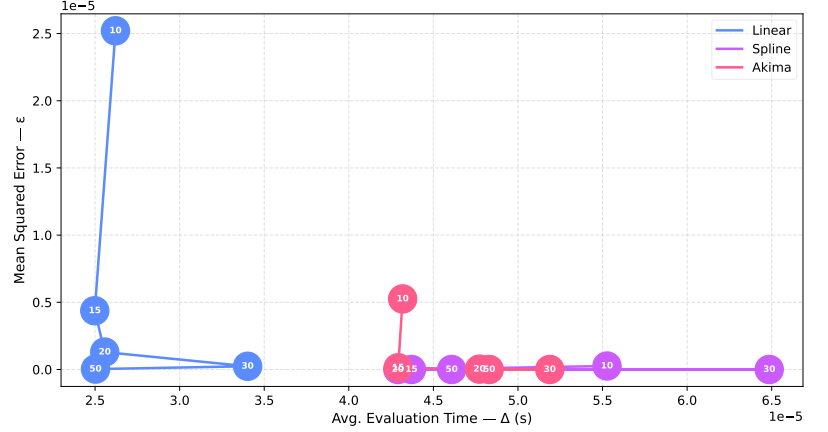


Figure 4: MSE vs. evaluation time by interpolation method.

	$n = 10$	$n = 15$	$n = 20$	$n = 30$	$n = 50$
Linear ε	$3.04 \cdot 10^{-5}$	$5.33 \cdot 10^{-6}$	$1.60 \cdot 10^{-6}$	$2.94 \cdot 10^{-7}$	$3.63 \cdot 10^{-8}$
Spline ε	$4.18 \cdot 10^{-7}$	$6.11 \cdot 10^{-9}$	$3.39 \cdot 10^{-10}$	$6.59 \cdot 10^{-12}$	$6.05 \cdot 10^{-14}$
Akima ε	$2.06 \cdot 10^{-6}$	$1.25 \cdot 10^{-7}$	$2.23 \cdot 10^{-8}$	$1.59 \cdot 10^{-9}$	$4.48 \cdot 10^{-11}$
Linear Δ	$5.4 \cdot 10^{-5}$	$5.5 \cdot 10^{-5}$	$5.7 \cdot 10^{-5}$	$6.8 \cdot 10^{-5}$	$8.3 \cdot 10^{-5}$
Spline Δ	$7.8 \cdot 10^{-5}$	$7.3 \cdot 10^{-5}$	$7.8 \cdot 10^{-5}$	$8.1 \cdot 10^{-5}$	$1.0 \cdot 10^{-4}$
Akima Δ	$7.1 \cdot 10^{-5}$	$7.5 \cdot 10^{-5}$	$9.0 \cdot 10^{-5}$	$9.7 \cdot 10^{-5}$	$9.8 \cdot 10^{-5}$

Table 4: Log-grid interpolation MSE (ε) and average evaluation time in seconds (Δ) by grid size.

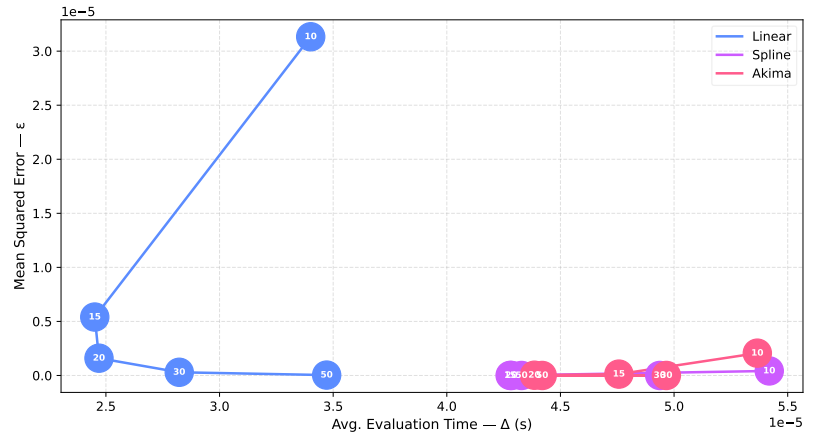


Figure 5: MSE vs. evaluation time by interpolation method (log grid).

Some results are still equal to question 2. For example, we see that while the initial performance of the linear interpolation is still poorer than that of other interpolations for $n = 10$, this difference quickly decays to zero as n increases. Also, the computational efficiency of the linear interpolation is now bigger compared to other methods than it was in the previous example. But the main difference here is probably that the log-spaced grid, in this case, performs on average worse than the uniform-spaced grid, contrary to the result we had in the previous question, specially for the linear interpolation with a low n . This suggests that we should decide which grid to use based on the information we have on the function we're trying to interpolate.

4 Question 4

In this question, we're asked to find the root of the polynomial $x^3 + 2x + 5$. We'll do this using two standard methods first: the bisection and the secant methods. These methods only require the function to be continuous, but not necessarily differentiable. As this polynomial is also differentiable, and we can easily compute its derivative, we can also use the Newton-Raphson method. In all methods, the error tolerance was set as $1e-8$. In order to find the root, however, item (a) first asks us to bracket the root, i.e., find two points in its domain whose image have different signs. If the function is continuous (which is the case), then we know that it has a root:

$$f(-2) = -7 \text{ and } f(2) = 17$$

We can apply all three methods and check its results:

Method	Value	Time (s)	Iterations
Bisection	-1.3283	0.0	27
Secant	-1.3283	0.0	6
Newton-Raphson	-1.3283	0.0	5

Table 5: Root-finding results: converged value, execution time, and iterations by method.

The results suggest that all functions have found the same root, which is indeed the root of this polynomial. As the function is relatively simple, all these methods have also been very quick in finding a solution. The bisection method, however, needed way more iterations to find the root. Naturally, the number of iterations depend on the size of the interval we set, and in my case it went from -2 to 2. Both the secant and the Newton-Raphson yielded a very similar number of iterations, with the Newton-Raphson having a small edge probably because of the advantage of the fact we're using its analytical derivative, instead of the numerical derivative from the secant method.